AI-Powered Travel Itinerary Planner

Project Title	AI-Powered Travel Itinerary Planner
Duration	4 Hours
Domain	Generative AI / Travel Tech
Tech Stack	Python, LangChain, Streamlit, GCP, Docker, Kubernetes

1. Project Title

AI-Powered Travel Itinerary Planner with GCP and ELK STACK

2. Duration

4-5 Hours

3. Introduction

This project is an intelligent web application designed to simplify travel planning. It utilizes Large Language Models (LLMs) to generate personalized, day-by-day travel itineraries based on user preferences such as city, duration, interests, and travel style. The application is containerized and deployed on Google Cloud Platform (GCP) using Kubernetes, demonstrating a complete end-to-end LLMOps pipeline.

4. Aim

The primary goal of this project is to build a robust, AI-driven agent that automates the creation of detailed travel plans. It aims to solve the problem of time-consuming research by aggregating real-time data on attractions, accommodations, and logistics into a cohesive schedule, while also providing a hands-on learning experience in deploying LLM applications with modern DevOps practices.

5. Dataset Used

Data Source: Real-time Web Search Data (Live APIs)

Primary Source: Google Serper API / Tavily Search API

Format: JSON (API Responses converted to Natural Language)

Description: Instead of a static dataset, this project leverages dynamic, real-time data fetched from the web to ensure travel recommendations are up-to-date and relevant.

6. Tools & Technologies

Programming Language: Python 3.10+

Frontend Reference: Streamlit

AI & Logic: LangChain (Agents & Tool Use), Groq (LLM Inference), OpenAI (Optional)

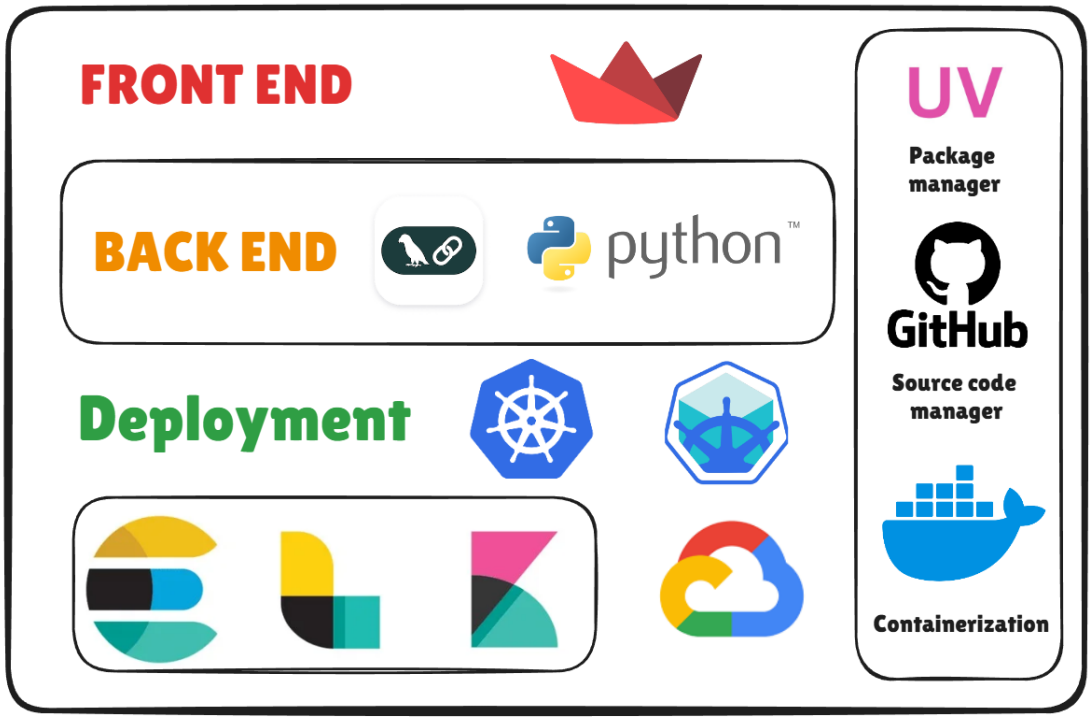
Cloud & DevOps:

• Containerization: Docker

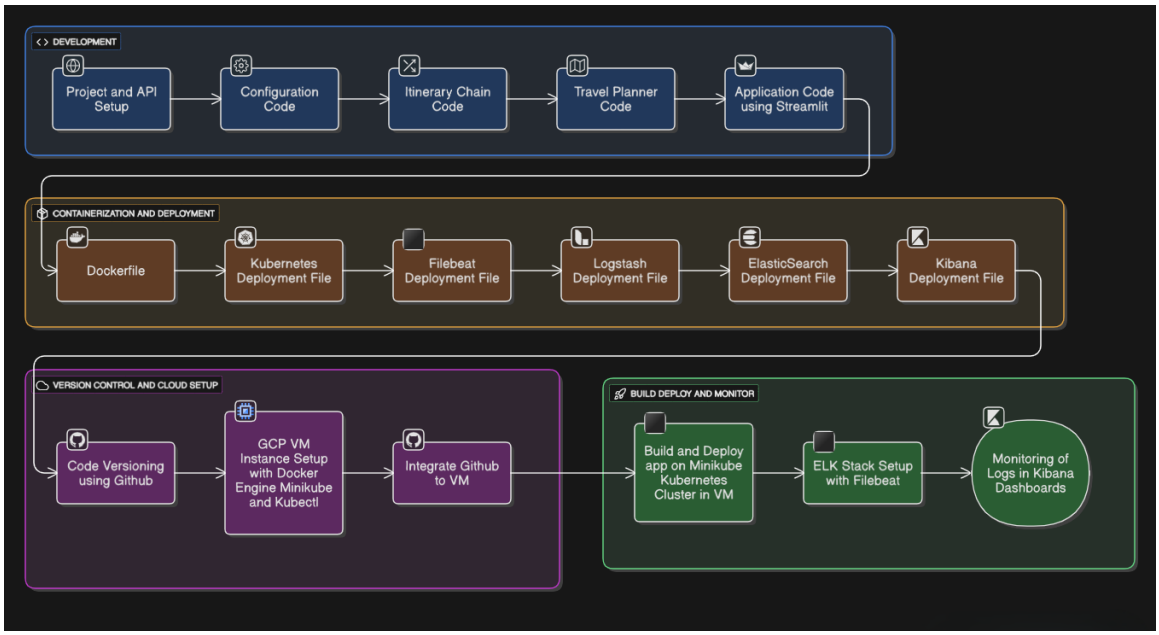
• Orchestration: Kubernetes (Minikube / GKE)

• Cloud Provider: Google Cloud Platform (Compute Engine)

Logging & Observability: ELK Stack (Elasticsearch, Logstash, Kibana, Filebeat)



7. Architecture Diagram



Workflow Description:

• User Interface: User inputs travel details (City, Interest, Days) via Streamlit.

• App Logic: The input is sent to the TravelPlanner Agent.

• Agent Action: The LangChain agent queries external tools (Serper/Tavily) for real-time info.

• LLM Processing: The LLM synthesizes the search results into a structured itinerary.

• Output: The final plan is displayed to the user.

• Background: Logs are shipped via Filebeat to the ELK stack for monitoring.

8. Key Steps / Modules

1. Environment Setup: Configuring the Python environment and API keys (Groq, Tavily, Serper).

2. Initial Development: Building the core TravelPlanner agent using LangChain and defining search tools.

3. Frontend Implementation: Creating the interactive UI with Streamlit.

4. Containerization: Writing the Dockerfile to package the application.

5. Cloud Provisioning: Setting up a VM Instance on Google Cloud Platform.

6. Orchestration Setup: Installing and configuring Minikube and verify Kubernetes status.

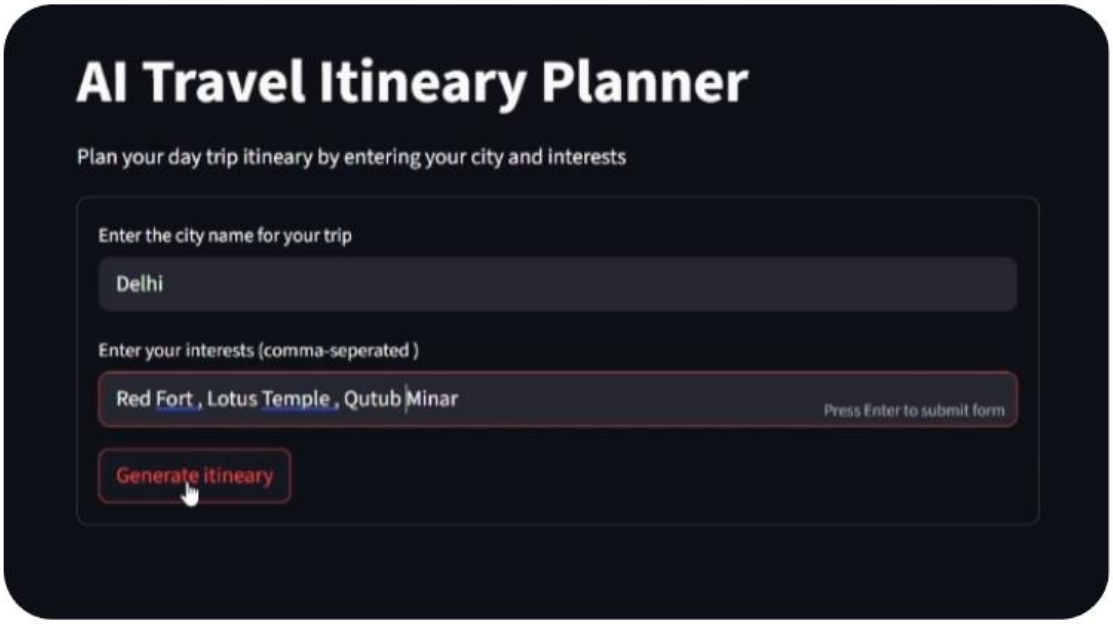
7. Deployment: Deploying the application pods and services to the Kubernetes cluster.

8. Observability Integration: Deploying the ELK stack (Elasticsearch, Logstash, Kibana) and Filebeat for log aggregation.

9. Learning Outcomes

- Develop AI Agents using LangChain and integrate them with external tools.
- Build interactive web interfaces for data science apps using Streamlit.
- Understand the principles of Containerization (Docker) for consistent environments.
- Master basic Kubernetes operations (Deployment, Services, Secrets) on a cloud VM.
- Implement production-grade Logging and Monitoring using the ELK stack.
- Navigate and manage cloud resources on Google Cloud Platform.

10. Screenshots



11. Optional Add-ons

- PDF Export: Allow users to download the itinerary as a PDF file.
- Map Integration: Visualize the daily route on an interactive map.
- Booking System: Integrate APIs to show real-time flight and hotel prices.
- User Accounts: Application of authentication to save and retrieve past itineraries.

12. Resource Links

- LangChain Documentation: https://www.langchain.com/